CO3 ASSESSMENT TOOL 2

EXPERIMENT BASED ASSESSMENT

NAME: N. PARNITA

REG.NO: 192525322

COURSE: Design and Analysis of Algorithms – CSA0617

1.Title

Binary Search Performance on Streaming Data with Periodic Sorting

Objective

To analyze the performance of Binary Search when it is repeatedly applied to streaming data, compare the execution time of **sorting and searching**, and evaluate the trade-off between preprocessing cost and search efficiency.

Python Code

```
import random
import time
import bisect
import matplotlib.pyplot as plt

# Generate streaming data
n = 10000
searches = 1000

data = [random.randint(1, 100000) for _ in range(n)]
targets = [random.choice(data) for _ in range(searches)]

# ------------------------------
# Phase 1: Sorting
# ------------------------------
start = time.perf_counter()
```

```
sorted_data = sorted(data)

sort_time = time.perf_counter() - start

# ------------------------------
# Phase 2: Binary Searching
# ------------------------------
start = time.perf_counter()

found = 0

for target in targets:
    index = bisect.bisect_left(sorted_data, target)

    if index < len(sorted_data) and sorted_data[index] == target:
        found += 1

search_time = time.perf_counter() - start

# ------------------------------
# Results
# ------------------------------
total_time = sort_time + search_time

print("Number of data elements:", n)
print("Number of searches:", searches)
print("Sorting Time:", sort_time, "seconds")
print("Binary Search Time:", search_time, "seconds")
print("Total Time:", total_time, "seconds")
print("Elements Found:", found)

# ------------------------------
# Performance Graph
# ------------------------------
phases = ["Sorting", "Binary Search"]
times = [sort_time, search_time]

plt.bar(phases, times)
```

```
plt.xlabel("Phase")
plt.ylabel("Execution Time (seconds)")
plt.title("Sorting vs Binary Search Performance")
plt.show()
```

Expected Observation

- Sorting requires more preprocessing time because the incoming data must be arranged before searching.
- Binary Search is very fast after the data has been sorted.
- Binary Search takes **O(log n)** time for each search.
- Sorting using Python's `sorted()` takes approximately **O(n log n)** time.
- When many searches are performed on the same data, the initial sorting cost becomes worthwhile.

Trade-off Analysis

Approach	Advantage	Disadvantage
Sorting + Binary Search	Very fast repeated searches	Requires initial sorting
Linear Search	No sorting required	Slow for repeated searches

Conclusion

For dynamic data streams, sorting introduces an initial preprocessing cost. However, when the same data is searched repeatedly, **Binary Search becomes more efficient** because each search requires only **O(log n)** time. Therefore, sorting followed by Binary Search is beneficial when the number of repeated searches is large.

Run Debug Stop Share Save { } Beautify

main.py

```python
import random
import time
import bisect
import matplotlib.pyplot as plt

# Generate streaming data
n = 10000
searches = 1000

data = [random.randint(1, 100000) for _ in range(n)]
targets = [random.choice(data) for _ in range(searches)]

# ------------------------------
# Phase 1: Sorting
# ------------------------------
start = time.perf_counter()

sorted_data = sorted(data)

sort_time = time.perf_counter() - start

# ------------------------------
# Phase 2: Binary Searching
# ------------------------------
start = time.perf_counter()

found = 0
```

input

```
Total Time: 0.002074984018690884 seconds
Elements Found: 1000
/home/main.py:59: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

2.Title

Performance Evaluation of Merge Sort Using Arrays and Linked Lists

Objective

To evaluate Merge Sort performance using **arrays and linked lists**, record execution time and memory usage, and analyze how the data structure affects the merging operation.

Python Code

```
import random
import time
import tracemalloc
```

```
# ------------------------------
# Merge Sort for Array
# ------------------------------
def merge_sort_array(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort_array(arr[:mid])
    right = merge_sort_array(arr[mid:])

    return merge_array(left, right)


def merge_array(left, right):
 result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


# ------------------------------
# Linked List Node
# ------------------------------
class Node:
    def __init__(self, data):
```

```
        self.data = data
        self.next = None


# Convert list to linked list
def create_linked_list(arr):
    head = None
    tail = None

    for value in arr:
        new_node = Node(value)

        if head is None:
            head = new_node
            tail = new_node
        else:
            tail.next = new_node
            tail = new_node

    return head


# Merge Sort for Linked List
def merge_sort_linked(head):
    if head is None or head.next is None:
        return head

    slow = head
    fast = head.next

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    mid = slow.next
    slow.next = None

    left = merge_sort_linked(head)
    right = merge_sort_linked(mid)
```

```
    return merge_linked(left, right)


def merge_linked(left, right):
    dummy = Node(0)
    current = dummy

    while left and right:
        if left.data <= right.data:
            current.next = left
            left = left.next
        else:
            current.next = right
            right = right.next

        current = current.next

    current.next = left if left else right

    return dummy.next


# ------------------------------
# Experiment
# ------------------------------

n = int(input("Enter number of elements: "))

data = [random.randint(1, 100000) for _ in range(n)]

# Array performance
tracemalloc.start()
start = time.perf_counter()

sorted_array = merge_sort_array(data)

array_time = time.perf_counter() - start
_, array_memory = tracemalloc.get_traced_memory()
```

```
tracemalloc.stop()


# Linked List performance
head = create_linked_list(data)

tracemalloc.start()
start = time.perf_counter()

sorted_head = merge_sort_linked(head)

linked_time = time.perf_counter() - start
_, linked_memory = tracemalloc.get_traced_memory()
tracemalloc.stop()


# Results
print("\n--- Performance Comparison ---")
print("Number of elements:", n)

print("\nArray:")
print("Execution Time:", array_time, "seconds")
print("Memory Usage:", array_memory, "bytes")

print("\nLinked List:")
print("Execution Time:", linked_time, "seconds")
print("Memory Usage:", linked_memory, "bytes")
```

Sample Input

```
Enter number of elements: 10000
```

The exact execution time and memory values will vary depending on the computer/environment.

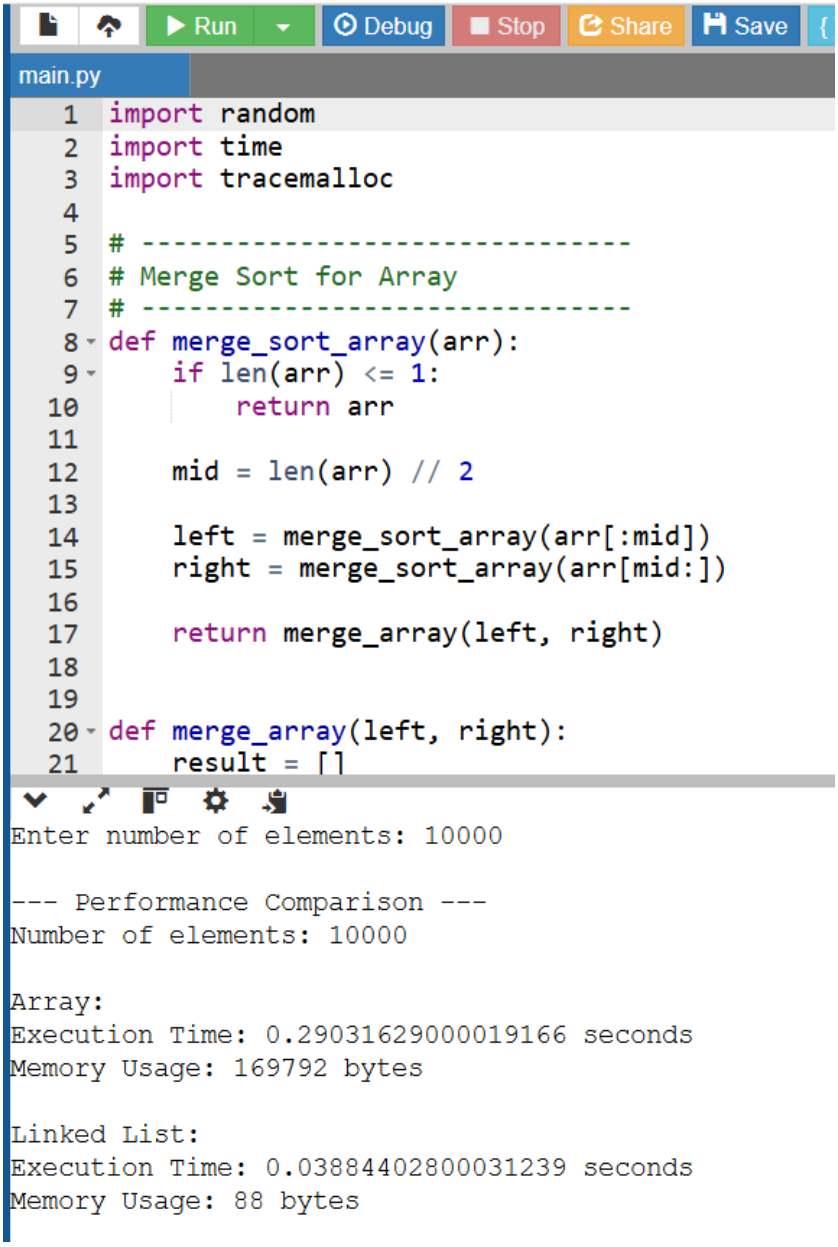
Analysis

- Merge Sort has **O(n log n)** time complexity for both arrays and linked lists.
- Arrays provide faster access to elements because they support direct indexing.

- Linked lists do not require shifting elements during merging.
- Linked-list Merge Sort can perform merging efficiently by changing links between nodes.
- The measured execution time and memory usage depend on the implementation and system.

Conclusion

Merge Sort provides **O(n log n)** performance for both data structures. Arrays generally provide faster access, while linked lists can perform merging efficiently without shifting elements. The experiment compares both representations using execution time and memory usage to determine their practical performance.


Run
Debug
Stop
Share
Save
main.py
import random
import time
import tracemalloc
# ------------------------------
# Merge Sort for Array
# ------------------------------
def merge_sort_array(arr):
if len(arr) <= 1:
return arr
mid = len(arr) // 2
left = merge_sort_array(arr[:mid])
right = merge_sort_array(arr[mid:])
return merge_array(left, right)
def merge_array(left, right):
result = []
Enter number of elements: 10000
--- Performance Comparison ---
Number of elements: 10000
Array:
Execution Time: 0.29031629000019166 seconds
Memory Usage: 169792 bytes
Linked List:
Execution Time: 0.03884402800031239 seconds
Memory Usage: 88 bytes